



```
9// Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .  
10P-MERGE ( $A, p, q, r$ )
```

First, let's use work/span analysis to get some intuition for why we need a parallel merge procedure. After all, it may seem as though there should be plenty of parallelism just by parallelizing MERGE-SORT without worrying about parallelizing the merge. But what would happen if the call to P-MERGE in line 10 of P-MERGE-SORT were replaced by a call to the serial MERGE procedure on page 36? Let's call the pseudocode so modified P-NAIVE-MERGE-SORT.

Let  $T_1(n)$  be the (worst-case) work of P-NAIVE-MERGE-SORT on an  $n$ -element subarray, where  $n = r - p + 1$  is the number of elements in  $A[p : r]$ , and let  $T_\infty(n)$  be the span. Because MERGE is serial with running time  $\Theta(n)$ , both its work and span are  $\Theta(n)$ . Since the serial projection of P-NAIVE-MERGE-SORT is exactly MERGE-SORT, its work is  $T_1(n) = \Theta(n \lg n)$ . The two recursive calls in lines 5 and 7 run in parallel, and so its span is given by the recurrence

$$\begin{aligned} T_\infty(n) &= T_\infty(n/2) + \Theta(n) \\ &= \Theta(n), \end{aligned}$$

by case 1 of the master theorem. Thus the parallelism of P-NAIVE-MERGE-SORT is  $T_1(n)/T_\infty(n) = \Theta(\lg n)$ , which is an unimpressive amount of parallelism. To sort a million elements, for example, since  $\lg 10^6 \approx 20$ , it might achieve linear speedup on a few processors, but it would not scale up to dozens of processors.

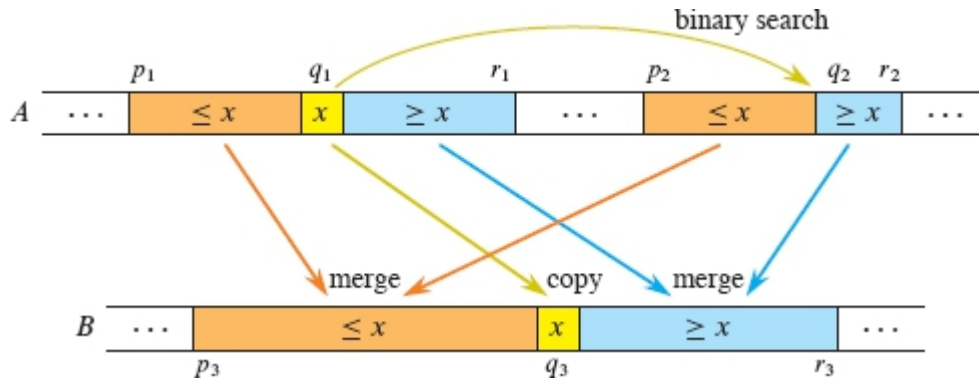
The parallelism bottleneck in P-NAIVE-MERGE-SORT is plainly the MERGE procedure. If we asymptotically reduce the span of merging, the master theorem dictates that the span of parallel merge sort will also get smaller. When you look at the pseudocode for MERGE, it may seem that merging is inherently serial, but it's not. We can fashion a parallel merging algorithm. The goal is to reduce the span of parallel merging asymptotically, but if we want an efficient parallel algorithm, we must ensure that the  $\Theta(n)$  bound on work doesn't increase.

Figure 26.6 depicts the divide-and-conquer strategy that we'll use in P-MERGE. The heart of the algorithm is a recursive auxiliary procedure P-MERGE-AUX that merges two sorted subarrays of an array  $A$  into a subarray of another array  $B$  in parallel. Specifically, P-MERGE-AUX merges  $A[p_1 : r_1]$  and  $A[p_2 : r_2]$  into subarray  $B[p_3 : r_3]$ , where  $r_3 = p_3 + (r_1 - p_1 + 1) + (r_2 - p_2 + 1) - 1 = p_3 + (r_1 - p_1) + (r_2 - p_2) + 1$ .

The key idea of the recursive merging algorithm in P-MERGE-AUX is to split each of the two sorted subarrays of  $A$  around a pivot  $x$ , such that all the elements in the lower part of each subarray are at most  $x$  and all the elements in the upper part of each subarray are at least  $x$ . The procedure can then recurse in parallel on two subtasks: merging the two lower parts, and merging the two upper parts. The trick is to find a pivot  $x$  so that the recursion is not too lopsided. We don't want a situation such as that in QUICKSORT on page 183, where bad partitioning elements lead to a dramatic loss of asymptotic efficiency. We could opt to partition around a random element, as RANDOMIZED-QUICKSORT on page 192 does, but because the input subarrays are sorted, P-MERGE-AUX can quickly determine a pivot that always works well.

Specifically, the recursive merging algorithm picks the pivot  $x$  as the middle element of the larger of the two input subarrays, which we can assume without loss of generality is  $A[p_1 : r_1]$ , since otherwise, the two subarrays can just switch roles. That is,  $x = A[q_1]$ , where  $q_1 = \lfloor (p_1 + r_1)/2 \rfloor$ . Because  $A[p_1 : r_1]$  is sorted,  $x$  is a median of the subarray elements: every element in  $A[p_1 : q_1 - 1]$  is no more than  $x$ , and every element in  $A[q_1 + 1 : r_1]$  is no less than  $x$ . Then the algorithm finds the "split point"  $q_2$  in the smaller subarray  $A[p_2 : r_2]$  such that all the elements in  $A[p_2 : q_2 - 1]$  (if any) are at most  $x$  and all the elements in  $A[q_2 : r_2]$  (if any) are at least  $x$ . Intuitively, the subarray  $A[p_2 : r_2]$  would still be sorted if  $x$  were inserted between  $A[q_2 - 1]$  and  $A[q_2]$  (although the algorithm doesn't do that). Since  $A[p_2 : r_2]$  is sorted, a minor variant of binary search (see Exercise 2.3-6) with  $x$  as the search key can find

the split point  $q_2$  in  $\Theta(\lg n)$  time in the worst case. As we'll see when we get to the analysis, even if  $x$  splits  $A[p_2 : r_2]$  badly— $x$  is either smaller than all the subarray elements or larger—we'll still have at least 1/4 of the elements in each of the two recursive merges. Thus the larger of the recursive merges operates on at most 3/4 elements, and the recursion is guaranteed to bottom out after  $\Theta(\lg n)$  recursive calls.



**Figure 26.6** The idea behind P-MERGE-AUX, which merges two sorted subarrays  $A[p_1 : r_1]$  and  $A[p_2 : r_2]$  into the subarray  $B[p_3 : r_3]$  in parallel. Letting  $x = A[q_1]$  (shown in yellow) be a median of  $A[p_1 : r_1]$  and  $q_2$  be a place in  $A[p_2 : r_2]$  such that  $x$  would fall between  $A[q_2 - 1]$  and  $A[q_2]$ , every element in the subarrays  $A[p_1 : q_1 - 1]$  and  $A[p_2 : q_2 - 1]$  (shown in orange) is at most  $x$ , and every element in the subarrays  $A[q_1 + 1 : r_1]$  and  $A[q_2 + 1 : r_2]$  (shown in blue) is at least  $x$ . To merge, compute the index  $q_3$  where  $x$  belongs in  $B[p_3 : r_3]$ , copy  $x$  into  $B[q_3]$ , and then recursively merge  $A[p_1 : q_1 - 1]$  with  $A[p_2 : q_2 - 1]$  into  $B[p_3 : q_3 - 1]$  and  $A[q_1 + 1 : r_1]$  with  $A[q_2 : r_2]$  into  $B[q_3 + 1 : r_3]$ .

Now let's put these ideas into pseudocode. We start with the serial procedure **FIND-SPLIT-POINT** ( $A, p, r, x$ ) on the next page, which takes as input a sorted subarray  $A[p : r]$  and a key  $x$ . The procedure returns a split point of  $A[p : r]$ : an index  $q$  in the range  $p \leq q \leq r + 1$  such that all the elements in  $A[p : q - 1]$  (if any) are at most  $x$  and all the elements in  $A[q : r]$  (if any) are at least  $x$ .

The **FIND-SPLIT-POINT** procedure uses binary search to find the split point. Lines 1 and 2 establish the range of indices for the search. Each time through the **while** loop, line 5 compares the middle element of the range with the search key  $x$ , and lines 6 and 7 narrow the search range to either the lower half or the upper half of the subarray,

depending on the result of the test. In the end, after the range has been narrowed to a single index, line 8 returns that index as the split point.

FIND-SPLIT-POINT ( $A, p, r, x$ )

```
1  $low = p$  // low end of search range
2  $high = r + 1$  // high end of search range
3 while  $low < high$  // more than one element?
4    $mid = \lfloor (low + high)/2 \rfloor$  // midpoint of range
5   if  $x \leq A[mid]$  // is answer  $q \leq mid$ ?
6      $high = mid$  // narrow search to  $A[low : mid]$ 
7   else  $low = mid + 1$  // narrow search to  $A[mid + 1 : high]$ 
8 return  $low$ 
```

Because FIND-SPLIT-POINT contains no parallelism, its span is just its serial running time, which is also its work. On a subarray  $A[p : r]$  of size  $n = r - p + 1$ , each iteration of the **while** loop halves the search range, which means that the loop terminates after  $\Theta(\lg n)$  iterations. Since each iteration takes constant time, the algorithm runs in  $\Theta(\lg n)$  (worst-case) time. Thus the procedure has work and span  $\Theta(\lg n)$ .

Let's now look at the pseudocode for the parallel merging procedure P-MERGE on the next page. Most of the pseudocode is devoted to the recursive procedure P-MERGE-AUX. The procedure P-MERGE itself is just a "wrapper" that sets up for P-MERGE-AUX. It allocates a new array  $B[p : r]$  to hold the output of P-MERGE-AUX in line 1. It then calls P-MERGE-AUX in line 2, passing the indices of the two subarrays to be merged and providing  $B$  as the output destination of the merged result, starting at index  $p$ . After P-MERGE-AUX returns, lines 3–4 perform a parallel copy of the output  $B[p : r]$  into the subarray  $A[p : r]$ , which is where P-MERGE-SORT expects it.

The P-MERGE-AUX procedure is the interesting part of the algorithm. Let's start by understanding the parameters of this recursive parallel procedure. The input array  $A$  and the four indices  $p_1, r_1, p_2, r_2$  specify the subarrays  $A[p_1 : r_1]$  and  $A[p_2 : r_2]$  to be merged. The array  $B$  and the index  $p_3$  indicate that the merged result should be stored into

$B[p_3 : r_3]$ , where  $r_3 = p_3 + (r_1 - p_1) + (r_2 - p_2) + 1$ , as we saw earlier. The end index  $r_3$  of the output subarray is not needed by the pseudocode, but it helps conceptually to name the end index, as in the comment in line 13.

The procedure begins by checking the base case of the recursion and doing some bookkeeping to simplify the rest of the pseudocode. Lines 1 and 2 test whether the two subarrays are both empty, in which case the procedure returns. Line 3 checks whether the first subarray contains fewer elements than the second subarray. Since the number of elements in the first subarray is  $r_1 - p_1 + 1$  and the number in the second subarray is  $r_2 - p_2 + 1$ , the test omits the two “+1’s.” If the first subarray is the smaller of the two, lines 4 and 5 switch the roles of the subarrays so that  $A[p_1, r_1]$  refers to the larger subarray for the balance of the procedure.

P-MERGE ( $A, p, q, r$ )

```

1  let  $B[p : r]$  be a new array           // allocate scratch array
2  P-MERGE-AUX ( $A, p, q, q + 1, r, B,$  // merge from  $A$  into  $B$ 
    $p$ )
3  parallel for  $i = p$  to  $r$            // copy  $B$  back to  $A$  in
                                       // parallel
4       $A[i] = B[i]$ 
```

P-MERGE-AUX ( $A, p_1, r_1, p_2, r_2, B, p_3$ )

```

1  if  $p_1 > r_1$  and  $p_2 > r_2$            // are both subarrays empty?
2      return
3  if  $r_1 - p_1 < r_2 - p_2$            // second subarray bigger?
4      exchange  $p_1$  with  $p_2$          // swap subarray roles
5      exchange  $r_1$  with  $r_2$ 
6   $q_1 = \lfloor (p_1 + r_1) / 2 \rfloor$        // midpoint of  $A[p_1 : r_1]$ 
7   $x = A[q_1]$                          // median of  $A[p_1 : r_1]$  is
                                       // pivot  $x$ 
```

```

     $q_2 = \text{FIND-SPLIT-POINT}(A, p_2, // \text{split } A[p_2 : r_2] \text{ around } x$ 
     $r_2, x)$ 
9   $q_3 = p_3 + (q_1 - p_1) + (q_2 - p_2)$            // where  $x$  belongs in  $B \dots$ 
10  $B[q_3] = x$                                      // ... put it there
11 // Recursively merge  $A[p_1 : q_1 - 1]$  and  $A[p_2 : q_2 - 1]$  into  $B[p_3 : q_3$ 
     $- 1]$ .
12 spawn P-MERGE-AUX ( $A, p_1, q_1 - 1, p_2, q_2 - 1, B, p_3$ )
13 // Recursively merge  $A[q_1 + 1 : r_1]$  and  $A[q_2 : r_2]$  into  $B[q_3 + 1 : r_3]$ .
14 spawn P-MERGE-AUX ( $A, q_1 + 1, r_1, q_2, r_2, B, q_3 + 1$ )
15 sync                                           // wait for spawns

```

We're now at the crux of P-MERGE-AUX: implementing the parallel divide-and-conquer strategy. As we continue our pseudocode walk, you may find it helpful to refer again to Figure 26.6.

First the divide step. Line 6 computes the midpoint  $q_1$  of  $A[p_1 : r_1]$ , which indexes a median  $x = A[q_1]$  of this subarray to be used as the pivot, and line 7 determines  $x$  itself. Next, line 8 uses the FIND-SPLIT-POINT procedure to find the index  $q_2$  in  $A[p_2 : r_2]$  such that all elements in  $A[p_2 : q_2 - 1]$  are at most  $x$  and all the elements in  $A[q_2 : r_2]$  are at least  $x$ . Line 9 computes the index  $q_3$  of the element that divides the output subarray  $B[p_3 : r_3]$  into  $B[p_3 : q_3 - 1]$  and  $B[q_3 + 1 : r_3]$ , and then line 10 puts  $x$  directly into  $B[q_3]$ , which is where it belongs in the output.

Next is the conquer step, which is where the parallel recursion occurs. Lines 12 and 14 each spawn P-MERGE-AUX to recursively merge from  $A$  into  $B$ , the first to merge the smaller elements and the second to merge the larger elements. The **sync** statement in line 15 ensures that the subproblems finish before the procedure returns.

There is no combine step, as  $B[p : r]$  already contains the correct sorted output.

## Work/span analysis of parallel merging

Let's first analyze the worst-case span  $T_\infty(n)$  of P-MERGE-AUX on input subarrays that together contain a total of  $n$  elements. The call to FIND-SPLIT-POINT in line 8 contributes  $\Theta(\lg n)$  to the span in the worst case, and the procedure performs at most a constant amount of additional serial work outside of the two recursive spawns in lines 12 and 14.

Because the two recursive spawns operate logically in parallel, only one of them contributes to the overall worst-case span. We claimed earlier that neither recursive invocation ever operates on more than  $3n/4$  elements. Let's see why. Let  $n_1 = r_1 - p_1 + 1$  and  $n_2 = r_2 - p_2 + 1$ , where  $n = n_1 + n_2$ , be the sizes of the two subarrays when line 6 starts executing, that is, after we have established that  $n_2 \leq n_1$  by swapping the roles of the two subarrays, if necessary. Since the pivot  $x$  is a median of  $A[p_1 : r_1]$ , in the worst case, a recursive merge involves at most  $n_1/2$  elements of  $A[p_1 : r_1]$ , but it might involve all  $n_2$  of the elements of  $A[p_2 : r_2]$ . Thus we can bound the number of elements involved in a recursive invocation of P-MERGE-AUX by

$$\begin{aligned} n_1/2 + n_2 &= (2n_1 + 4n_2)/4 \\ &\leq (3n_1 + 3n_2)/4 \text{ (since } n_2 \leq n_1) \\ &= 3n/4, \end{aligned}$$

proving the claim.

The worst-case span of P-MERGE-AUX can therefore be described by the following recurrence:

$$T_\infty(n) = T_\infty(3n/4) + \Theta(\lg n). \quad (26.7)$$

Because this recurrence falls under case 2 of the master theorem with  $k = 1$ , its solution is  $T_\infty(n) = \Theta(\lg^2 n)$ .

Now let's verify that the work  $T_1(n)$  of P-MERGE-AUX on  $n$  elements is linear. A lower bound of  $\Omega(n)$  is straightforward, since each of the  $n$  elements is copied from array  $A$  to array  $B$ . We'll show that  $T_1(n) = O(n)$  by deriving a recurrence for the worst-case work. The



binary search in line 8 costs  $\Theta(\lg n)$  in the worst case, which dominates the other work outside of the recursive spawns. For the recursive spawns, observe that although lines 12 and 14 might merge different numbers of elements, the two recursive spawns together merge at most  $n - 1$  elements (since  $x = A[q]$  is not merged). Moreover, as we saw when analyzing the span, a recursive spawn operates on at most  $3n/4$  elements. We therefore obtain the recurrence

$$T_1(n) = T_1(\alpha n) + T_1((1 - \alpha)n) + \Theta(\lg n), \quad (26.8)$$

where  $\alpha$  lies in the range  $1/4 \leq \alpha \leq 3/4$ . The value of  $\alpha$  can vary from one recursive invocation to another.

We'll use the substitution method (see Section 4.3) to prove that the above recurrence (26.8) has solution  $T_1(n) = O(n)$ . (You could also use the Akra-Bazzi method from Section 4.7.) Assume that  $T_1(n) \leq c_1 n - c_2 \lg n$  for some positive constants  $c_1$  and  $c_2$ . Using the properties of logarithms on pages 66–67—in particular, to deduce that  $\lg \alpha + \lg(1 - \alpha) = -\Theta(1)$ —substitution yields

$$\begin{aligned} T_1(n) &\leq (c_1 \alpha n - c_2 \lg(\alpha n)) + (c_1(1 - \alpha)n - c_2 \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1(\alpha + (1 - \alpha))n - c_2(\lg(\alpha n) + \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1 n - c_2(\lg \alpha + \lg n + \lg(1 - \alpha) + \lg n) + \Theta(\lg n) \\ &= c_1 n - c_2 \lg n - c_2(\lg n + \lg \alpha + \lg(1 - \alpha)) + \Theta(\lg n) \\ &= c_1 n - c_2 \lg n - c_2(\lg n - \Theta(1)) + \Theta(\lg n) \\ &\leq c_1 n - c_2 \lg n, \end{aligned}$$

if we choose  $c_2$  large enough that the  $c_2(\lg n - \Theta(1))$  term dominates the  $\Theta(\lg n)$  term for sufficiently large  $n$ . Furthermore, we can choose  $c_1$  large enough to satisfy the implied  $\Theta(1)$  base cases of the recurrence, completing the induction. The lower and upper bounds of  $\Omega(n)$  and  $O(n)$  give  $T_1(n) = \Theta(n)$ , asymptotically the same work as for serial merging.

The execution of the pseudocode in the P-MERGE procedure itself does not add asymptotically to the work and span of P-MERGE-AUX.

The **parallel for** loop in lines 3–4 has  $\Theta(\lg n)$  span due to the loop control, and each iteration runs in constant time. Thus the  $\Theta(\lg^2 n)$  span of P-MERGE-AUX dominates, yielding  $\Theta(\lg^2 n)$  span overall for P-MERGE. The **parallel for** loop contains  $\Theta(n)$  work, matching the asymptotic work of P-MERGE-AUX and yielding  $\Theta(n)$  work overall for P-MERGE.

### Analysis of parallel merge sort

The “heavy lifting” is done. Now that we have determined the work and span of P-MERGE, we can analyze P-MERGE-SORT. Let  $T_1(n)$  and  $T_\infty(n)$  be the work and span, respectively, of P-MERGE-SORT on an array of  $n$  elements. The call to P-MERGE in line 10 of P-MERGE-SORT dominates the costs of lines 1–3, for both work and span. Thus we obtain the recurrence

$$T_1(n) = 2T_1(n/2) + \Theta(n)$$

for the work of P-MERGE-SORT, and we obtain the recurrence

$$T_\infty(n) = T_\infty(n/2) + \Theta(\lg^2 n)$$

for its span. The work recurrence has solution  $T_1(n) = \Theta(n \lg n)$  by case 2 of the master theorem with  $k = 0$ . The span recurrence has solution  $T_\infty(n) = \Theta(\lg^3 n)$ , also by case 2 of the master theorem, but with  $k = 2$ .

Parallel merging gives P-MERGE-SORT a parallelism advantage over P-NAIVE-MERGE-SORT. The parallelism of P-NAIVE-MERGE-SORT, which calls the serial MERGE procedure, is only  $\Theta(\lg n)$ . For P-MERGE-SORT, the parallelism is

$$\begin{aligned} T_1(n)/T_\infty(n) &= \Theta(n \lg n)/\Theta(\lg^3 n) \\ &= \Theta(n/\lg^2 n), \end{aligned}$$

which is much better, both in theory and in practice. A good implementation in practice would sacrifice some parallelism by coarsening the base case in order to reduce the constants hidden by the

asymptotic notation. For example, you could switch to an efficient serial sort, perhaps quicksort, when the number of elements to be sorted is sufficiently small.

## Exercises

### 26.3-1

Explain how to coarsen the base case of P-MERGE.

### 26.3-2

Instead of finding a median element in the larger subarray, as P-MERGE does, suppose that the merge procedure finds a median of all the elements in the two sorted subarrays using the result of Exercise 9.3-10. Give pseudocode for an efficient parallel merging procedure that uses this median-finding procedure. Analyze your algorithm.

### 26.3-3

Give an efficient parallel algorithm for partitioning an array around a pivot, as is done by the PARTITION procedure on page 184. You need not partition the array in place. Make your algorithm as parallel as possible. Analyze your algorithm. (*Hint:* You might need an auxiliary array and might need to make more than one pass over the input elements.)

### 26.3-4

Give a parallel version of FFT on page 890. Make your implementation as parallel as possible. Analyze your algorithm.

### ★ 26.3-5

Show how to parallelize SELECT from Section 9.3. Make your implementation as parallel as possible. Analyze your algorithm.

---

## Problems

### 26-1 *Implementing parallel loops using recursive spawning*